

Introduction to Software Architecture

Shah Ali Newaj Topu

The Problem – Complexity in Modern Software

- Key Challenges:
 - Increasing business complexity and changing requirements
 - Communication gap between business stakeholders and developers
 - Systems that are hard to maintain, understand, or extend
- Why Traditional Approaches Fall Short:
 - Overemphasis on technical architecture over business logic
 - Bloated codebases with mixed concerns
 - Lack of shared language and alignment

What Is Domain-Driven Design

- **Definition:**
 - A design approach that emphasizes deep connection between the business domain and the software model.
- **Core Idea:**
 - Tackle complexity in the heart of software by aligning code with the domain through modeling and collaboration.
- **Key Focus:**
 - Collaborating closely with domain experts
 - Modeling rich, expressive domains
 - Reflecting business knowledge in the software structure

Goals of DDD – Focus on Core Domain

- Why Core Domain Matters:
 - Core domain is the source of competitive advantage
 - Not all parts of the system deserve equal attention
- DDD Helps By:
 - Isolating the core logic from generic or infrastructure concerns
 - Enabling innovation in the areas that matter most
 - Structuring the system around the business value

Benefits of DDD

- Technical Benefits:
 - Modular, loosely coupled codebases
 - Easier to refactor and extend
 - Better testability and maintainability
- Business Benefits:
 - Improved collaboration with domain experts
 - Shared understanding via Ubiquitous Language
 - Higher confidence in reflecting business needs
- Organizational Benefits:
 - More effective teams
 - Better agility and product evolution

When to Use DDD - And When Not To

- Best Use Cases:
 - Complex business domains
 - Rapidly changing rules or logic
 - Need for long-term maintainability
- When DDD May Be Overkill:
 - Simple CRUD apps with little business logic
 - Short-lived or throwaway projects
 - Teams without access to domain experts
- DDD Is Not a Silver Bullet:
 - It requires deep collaboration, investment, and discipline

Misconceptions About DDD

- Common Myths:

- DDD is only for big companies
- DDD means overengineering
- DDD is all about aggregates and entities
- You must implement everything from the Blue Book

- Clarifications:

- DDD is not about complexity, it's about managing it
- Tactical patterns (aggregates, entities) are tools — not the goal
- Focus is on modeling and strategic thinking

A Quick History – Eric Evans and the Blue Book

- **Origins:**
 - Introduced by Eric Evans in 2003
 - "Domain-Driven Design: Tackling Complexity in the Heart of Software"
- **Impact:**
 - Influenced modern software architecture practices
 - Inspired many complementary approaches (e.g., CQRS, Event Sourcing)
- **Legacy:**
 - Still relevant in microservices, distributed systems, and modern enterprise software

Key Terminologies at a Glance

- **Domain:** The sphere of knowledge and activity around which the application logic revolves.
- **Model:** A system of abstractions that describes selected aspects of a domain.
- **Ubiquitous Language:** Shared language used by developers and domain experts.
- **Entity:** Object with a distinct identity that runs through time.
- **Value Object:** Object that describes characteristics without identity.
- **Aggregate:** A cluster of domain objects treated as a single unit.
- **Repository:** Mechanism to access aggregates.
- **Bounded Context:** Explicit boundary where a particular model is defined and applicable.

Domains

- Core Domain: The strategic heart of the business
- Supporting Subdomain: Helps core domain, but not unique
- Generic Subdomain: Common functionality (e.g., payments, logging)

Domain Types Examples

Online Learning Platform

Subdomain Type	Subdomain	Purpose	Strategic Value
Core Domain	Course Management	Designing, organizing, and delivering learning content	High – differentiates business
Supporting Subdomain	Instructor Scheduling	Assigning instructors to courses, managing conflicts	Medium – necessary but not core
Generic Subdomain	Authentication & Billing	User login, payment gateway integration	Low – commodity services

Handson 1

Identify Domains.

Ubiquitous Language

- A shared language developed by the team (devs + domain experts)
- Embedded in code, documentation, tests, and conversations
- Reduces miscommunication and improves clarity
- Evolves with domain understanding

Ubiquitous Language – Example

Term in Domain Conversation	Misaligned Code Term	Refactored to Match Ubiquitous Language
"Cart"	TempOrder	ShoppingCart
"Checkout"	SubmitCart()	CheckoutCart()
"Customer"	UserProfile	Customer
"Add Item to Cart"	InsertLineItem(int id)	AddProductToCart(ProductId)
"Place Order"	ConfirmPurchase ()	PlaceOrder()

Bounded Context

- A semantic boundary where a particular model is defined and applicable
- Same terms may have different meanings in different contexts
- Each bounded context has:
- Its own Ubiquitous Language
- Clear integration contracts

Bounded Context – Example

Different Meanings of "Order" in Two Bounded Contexts

Aspect	Ordering Context	Shipping Context
Purpose	Customer places and pays for an order	Logistics handles delivery of packages
Entity: Order	Includes OrderItems, PaymentInfo, Total	Includes TrackingNumber, ShippingDate
Lifecycle	Created during checkout	Created after payment is confirmed
Ubiquitous Term	"Place Order", "Calculate Total"	"Ship Order", "Assign Courier"
Data Ownership	Pricing, Discounts, Payment Status	Warehouse Location, Tracking, Courier Details

Identifying Bounded Contexts in a Large System

- Look for:
 - Different domain models or meanings for the same term
 - Organizational divisions
 - Data ownership boundaries
 - Change rate divergence
- Use Event Storming or domain workshops

Context Maps and Their Relationship Types

- Context Map: A diagram of all bounded contexts and their relationships
- Types of relationships:
 - Partnership
 - Shared Kernel
 - Customer–Supplier
 - Conformist
 - Anticorruption Layer
 - Open Host Service / Published Language

Relationship Types Example

- Partnership: Booking & Pricing teams collaborate, evolve together
- Shared Kernel: Common model (e.g., FareRule) shared between Pricing & Payment
- Customer–Supplier: Payment depends on Pricing API; Pricing team controls model
- Conformist: Payment conforms to Pricing without negotiation
- Anticorruption Layer: Protect Booking from inconsistent external Airline APIs
- Open Host Service: Notification exposes clean API for other contexts to consume

Handson 2: Bounded Context Mapping

Objective: Break the domain into bounded contexts.

Format: Context map drawing on whiteboard or digital canvas.

Bonus: Discuss upstream/downstream relationships and potential integration contracts.